Documento de arquitectura

PWA offline-first para reportes de servicios técnicos

Proyecto integrador · UTM
Versión: 1.0
Fecha: 28 de octubre de 2026
Entregable: Día 1 — Análisis y decisión arquitectónica (20 % de la rúbrica)

Resumen ejecutivo

Un técnico de servicios llega a una zona rural sin señal, atiende el reporte y necesita registrarlo antes de volver a la ciudad. Si la aplicación depende de la red, el reporte se pierde, se escribe en papel o se captura dos veces. La arquitectura elegida separa la captura del transporte: el navegador guarda primero en IndexedDB y una cola idempotente envía después, cuando hay conexión. PostgreSQL conserva la copia definitiva en el servidor.

La alternativa PWA obtuvo **4.78 de 5** en la matriz ponderada. La multiplataforma quedó en 3.78 y la nativa en 2.80. La diferencia no proviene de una sola casilla: la PWA es la única opción que combina almacenamiento local real, una base de código y distribución por URL sin tiendas.

---

1. Contexto y problema

El escenario es una empresa de servicios (internet, electricidad, refrigeración) cuyos técnicos trabajan en rancherías y comunidades con cobertura intermitente o nula. Los dispositivos son heterogéneos: Android de gama baja, iPhone y alguna computadora en oficina. No hay presupuesto para desarrollar dos aplicaciones ni para mantener cuentas de desarrollador en dos tiendas.

El flujo crítico ocurre en campo y tiene tres momentos:

1. El técnico llena el reporte y pulsa **Guardar**.
2. La aplicación confirma que el reporte quedó almacenado en el dispositivo.
3. Cuando vuelve la señal, el reporte llega al servidor una sola vez, aunque el envío se reintente.

Cualquier decisión que haga depender el paso 1 de la red incumple el requerimiento principal. El paso 3 introduce el problema de la idempotencia: una respuesta perdida no distingue entre "no llegó" y "llegó pero no me enteré", así que el cliente debe poder reenviar sin duplicar.

Restricciones del proyecto

Restricción	Consecuencia
Zona rural sin cobertura	La captura no puede esperar a la red.
Android e iOS	La solución debe funcionar en Chromium y WebKit.
Un solo equipo de desarrollo	Se prefiere una base de código y pocas dependencias.
Dos semanas de calendario	El despliegue no puede depender de trámites de tienda.
Presupuesto cero	Se usan planes gratuitos y software open source.
Evaluación con rúbrica	Cada decisión debe poder demostrarse con código y evidencia.

---

2. Requerimientos

Requerimientos funcionales

ID	Requerimiento	Prioridad	Criterio de aceptación
R1	Registrar un reporte sin red y no perderlo	Crítico	Con la red desconectada, el reporte aparece en la lista como pendiente y sigue ahí después de recargar.
R2	Sincronizar al recuperar conexión sin duplicar	Crítico	Reenviar el mismo UUID devuelve 200 y no crea una segunda fila.
R3	Instalarse desde una URL en Android e iOS	Crítico	Chrome Android muestra la instalación; Safari iOS permite agregar a inicio con una guía.
R4	Coste y tiempo de desarrollo bajos	Alto	Una base de código, sin tiendas y con despliegue por `git push`.
R5	Funcionar en hardware heterogéneo	Alto	La interfaz responde entre 320 px y 1440 px sin desbordamiento.
R6	Capturar GPS y permitir una referencia manual	Medio	El botón GPS obtiene coordenadas; si se deniega el permiso, el campo escrito sigue disponible.
R7	Actualizar sin revisión de tienda	Medio	Un despliegue nuevo se publica con una URL; el Service Worker actualiza el shell.

Requerimientos no funcionales

Atributo	Meta	Verificación
Disponibilidad de captura	100 % local, sin dependencia de la API	Prueba en modo avión con recarga.
Integridad	Cero duplicados por reintento	`ON CONFLICT (id) DO UPDATE` y prueba del mismo UUID.
Seguridad	HTTPS, CORS con lista permitida y consultas parametrizadas	Revisión de configuración y código.
Rendimiento	Shell por debajo de 300 KiB precacheados	Salida de `npm run build`.
Accesibilidad	Contraste AA, etiquetas y foco visible	Lighthouse y revisión manual.
Mantenibilidad	JS nativo, sin ORM y con módulos separados	Estructura del repositorio.

---

3. Alternativas evaluadas

Se compararon cinco paradigmas. La escala de cada criterio va de 1 a 5, donde 5 es la mejor nota para el proyecto.

3.1 Aplicación web tradicional

El servidor genera el HTML y el navegador solo lo pinta. Es la opción más barata y rápida de desplegar, pero cada pantalla exige una petición. Sin red no hay formulario, no hay lista y no hay captura. Incumple R1 de forma directa. Se descarta.

3.2 Aplicación orientada a servicios (SOA/API)

El cliente es delgado: la interfaz consume endpoints y no conserva estado propio. Es una arquitectura correcta para un back-office con red estable y permite reutilizar la API en otros clientes. En campo, el `fetch` falla y el cliente se queda sin flujo de trabajo. El problema no es SOA como diseño de servidor, sino el acoplamiento síncrono entre la captura y la red. Se descarta como arquitectura completa; la API se conserva dentro de la solución PWA.

3.3 Aplicación nativa

Swift en iOS y Kotlin en Android ofrecen el mejor acceso al hardware y un almacenamiento local maduro. El coste es alto: dos bases de código, dos pipelines y dos publicaciones. Además, la instalación pasa por App Store y Play Store, con revisión y cuentas de desarrollador. Incumple R4 y R7 para el presupuesto y el plazo disponibles. Se descarta.

3.4 Aplicación multiplataforma

Flutter o React Native reducen el problema a una base de código y conservan buen acceso al hardware. Sin embargo, siguen necesitando empaquetado, firma y distribución por tiendas. Añaden un runtime y un ecosistema de plugins que este proyecto no necesita. Queda segunda en la matriz, pero por debajo de la PWA en coste y despliegue. Se descarta.

3.5 PWA

Una aplicación web con manifest, Service Worker y almacenamiento local. Se instala desde una URL, funciona sin red después de la primera visita y se actualiza como cualquier sitio. Su punto débil es el acceso a APIs del dispositivo: GPS y cámara sí, Bluetooth, NFC o contactos no. El proyecto solo requiere GPS, así que la pérdida es marginal. Es la alternativa elegida.

---

4. Matriz de decisión ponderada

Peso total = 100 %. Puntuación final = Σ (peso × nota).

Criterio (peso)	Web tradicional	SOA / API	Nativa	Multiplataforma	PWA
Capacidad offline real (30 %)	1	1	5	5	**5**
Coste y tiempo de desarrollo (20 %)	5	4	1	3	**5**
Despliegue e instalación (20 %)	5	5	1	2	**5**
Independencia de hardware (15 %)	5	5	1	4	**5**
Acceso a APIs del móvil (15 %)	1	2	5	4.5	**3.5**
Total ponderado	**3.20**	**3.15**	**2.80**	**3.78**	**4.78**

Cálculo verificable:

```
Web:       0.30(1) + 0.20(5) + 0.20(5) + 0.15(5) + 0.15(1)   = 3.20
SOA:       0.30(1) + 0.20(4) + 0.20(5) + 0.15(5) + 0.15(2)   = 3.15
Nativa:    0.30(5) + 0.20(1) + 0.20(1) + 0.15(1) + 0.15(5)   = 2.80
Multi:     0.30(5) + 0.20(3) + 0.20(2) + 0.15(4) + 0.15(4.5) = 3.78
PWA:       0.30(5) + 0.20(5) + 0.20(5) + 0.15(5) + 0.15(3.5) = 4.78
```

> **Nota de control:** la primera versión del plan traía totales (2.80 / 2.85 / 3.05 / 3.85 / 4.75) que no coincidían con los pesos declarados. Aquí se recalcularon. El orden no cambia: la PWA gana y la nativa queda última por coste y distribución.

Lectura de la matriz

La PWA no gana por el acceso al hardware, donde saca 3.5. Gana porque el criterio con más peso, la capacidad offline, vale 30 %, y porque despliegue y coste suman otro 40 %. La nativa obtiene un 5 en offline y en APIs, pero un 1 en coste y despliegue la hunde. La multiplataforma corrige parte del coste, no la distribución por tiendas.

---

5. Justificación por criterio

Capacidad offline (30 %). La web tradicional y el cliente SOA puro asumen red. La nativa, la multiplataforma y la PWA pueden guardar localmente. En la PWA, el Service Worker conserva el shell y IndexedDB conserva los datos; la cola de sincronización decide cuándo enviar. La app abre y captura con la red apagada.

Coste y tiempo (20 %). Una sola base de código HTML, CSS y JavaScript. No hay Swift, Kotlin, Gradle ni Xcode. Las dependencias de producción del frontend se reducen a Dexie; el resto son herramientas de build y Workbox. El backend usa Express y `pg`, sin ORM.

Despliegue e instalación (20 %). La PWA se publica con un `push` y se instala desde una URL. No hay revisión de tienda ni firma de binarios. La nativa exige cuenta de desarrollador, compilación por plataforma y un proceso de revisión que puede rechazar o retrasar una corrección.

Independencia de hardware (15 %). Cualquier dispositivo con un navegador moderno puede abrir la app. La nativa se ata a su plataforma y la multiplataforma a su runtime. La PWA está diseñada para funcionar en Android, iOS y escritorio con el mismo código; Android se verificó en hardware y iOS queda documentado como compatibilidad no probada.

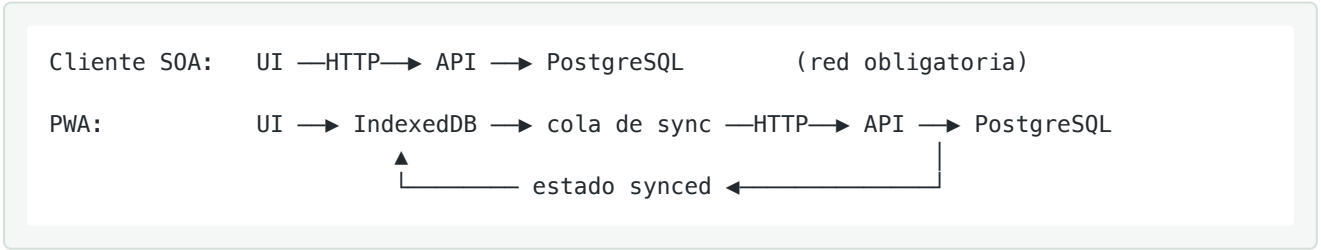
APIs del móvil (15 %). Aquí la PWA pierde. GPS, cámara y almacenamiento están disponibles; Bluetooth, NFC, contactos y algunos sensores no. El proyecto solo necesita geolocalización, que funciona sobre HTTPS y con permiso del usuario. Reconocer esta limitación es parte de la decisión, no un detalle que se oculta.

---

6. SOA frente a PWA

Una arquitectura orientada a servicios implementa clientes delgados: la interfaz solo pinta lo que responde la API. Es una decisión correcta cuando la red es estable, porque centraliza reglas y evita duplicar lógica. En campo, el mismo diseño se convierte en un punto único de fallo: si el `fetch` no responde, el técnico no puede capturar.

La PWA no niega SOA. Lo conserva en el servidor y añade una capa de resiliencia en el cliente.



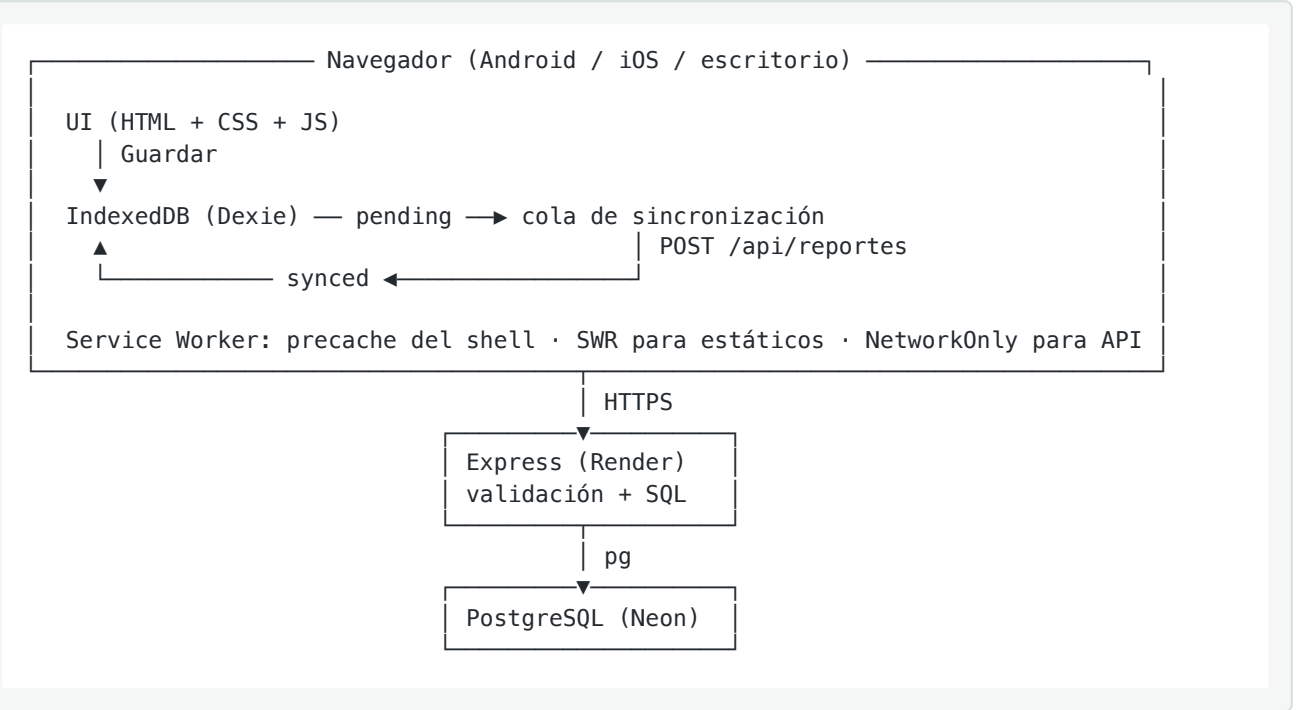
Tres piezas hacen la diferencia:

1. **Service Worker.** Sirve el shell desde caché. La app abre sin red y sin depender de la API.
2. **IndexedDB.** Guarda el reporte con un UUID generado en el cliente. La captura no depende del transporte.
3. **Cola idempotente.** Reintenta cuando hay red. El UUID viaja desde el cliente y el servidor hace `ON CONFLICT`, así que reintentar no duplica.

El resultado es **offline-first**, no "offline tolerante". En una app tolerante al offline, la red sigue siendo el camino principal y el modo desconectado es una excepción. Aquí el camino principal es local: la red acelera la sincronización, pero no habilita la captura.

---

7. Arquitectura propuesta



Componentes

Capa	Tecnología	Responsabilidad
Interfaz	HTML5 semántico, CSS3 y JS nativo	Formulario, listado, GPS, estados de red e instalación.
Build	Vite 7	Empaquetado, hashes de producción y proxy `/api` en local.
PWA	`vite-plugin-pwa` con `injectManifest`	Inyecta el manifiesto de precache en un `sw.js` propio.
Service Worker	Workbox 7	Precaching, Stale-While-Revalidate, `NetworkOnly` para la API y `offline.html`.
Almacenamiento local	Dexie 4 sobre IndexedDB	Reportes con estado `pending`, `synced` o `error`.
Sincronización	`sync.js`	Cola de envío, guardia de concurrencia y actualización de estado.
API	Node 20, Express 5 y `pg`	Validación, CORS y persistencia idempotente.
Base de datos	PostgreSQL 16	Fuente de verdad del servidor.
Infraestructura local	Docker Compose	Frontend, API y base de datos con un comando.
Producción	Vercel, Render y Neon	Estáticos, API y base de datos gestionada.

Decisiones y alternativas descartadas

Decisión	Alternativa descartada	Motivo
JS nativo con Vite	React, Vue o Svelte	La app tiene dos vistas y un formulario. Un framework añadiría runtime y estado global sin resolver ningún problema del proyecto.
`injectManifest`	`generateSW`	El ciclo `install`, `activate` y `fetch` debe poder explicarse y modificarse. `generateSW` lo oculta.
Dexie	IndexedDB nativo	La consulta por `syncStatus` y las actualizaciones puntuales se escriben en una línea. El API nativo exige más código para lo mismo.
Express + `pg`	ORM	Hay dos endpoints y una tabla. El SQL explícito hace visible la idempotencia.
Docker Compose	Instalación manual	Reproduce Node, PostgreSQL y versiones con un comando; evita "en mi máquina funciona".
UUID en el cliente	ID autogenerado en el servidor	Permite reintentar sin duplicar y guardar antes de la red.
Sincronización por evento `online` y botón	Background Sync API	Safari iOS no la soporta. La app necesita un camino que funcione en ambos sistemas.
Fuente autoalojada	Google Fonts	La app no debe depender de un tercero para verse igual sin red.

Flujo de captura

1. El técnico completa técnico, cliente, servicio, diagnóstico y, opcionalmente, ubicación y GPS.
2. Al enviar, el formulario valida los cuatro campos obligatorios.
3. Se genera `crypto.randomUUID()` y se arma el objeto con `syncStatus: 'pending'`.
4. Dexie escribe el reporte en IndexedDB. La interfaz cambia a la lista y confirma el guardado.
5. Si hay red, se dispara la sincronización inmediata. Si no, el reporte queda en cola y se avisa al técnico.

El orden importa: primero se escribe en disco, después se intenta la red. Un fallo del servidor no puede impedir que el reporte quede guardado.

Flujo de sincronización

1. Se leen los reportes con `syncStatus = 'pending'`.
2. Por cada uno se hace `POST /api/reportes` con el UUID local.
3. El servidor responde 201 si insertó y 200 si actualizó el mismo registro.
4. El cliente marca `synced`, guarda `syncedAt` y limpia `lastError`.
5. Un 4xx marca `error` y no se reintenta en bucle, porque el problema son los datos.
6. Un fallo de red deja el reporte en `pending` y detiene la cola si la conexión se perdió.

La función `syncPending()` mantiene una sola ejecución a la vez. Si el evento `online`, el botón manual y el arranque coinciden, no se envían dos ráfagas en paralelo.

Idempotencia en el servidor

```
INSERT INTO reportes (id, tecnico, cliente, servicio, descripcion, ubicacion, lat, lng,
estado, fecha_creacion)
VALUES ($1, $2, $3, $4, $5, $6, $7, $8, COALESCE($9, 'reportado'), COALESCE($10, now()))
ON CONFLICT (id) DO UPDATE SET
  tecnico = EXCLUDED.tecnico,
  cliente = EXCLUDED.cliente,
  servicio = EXCLUDED.servicio,
  descripcion = EXCLUDED.descripcion,
  ubicacion = EXCLUDED.ubicacion,
  lat = EXCLUDED.lat,
  lng = EXCLUDED.lng,
  estado = EXCLUDED.estado,
  actualizado_en = now()
RETURNING *, (xmax = 0) AS insertado;
```

El UUID es la clave primaria. Un reintento con el mismo identificador actualiza la fila existente y responde 200; un identificador nuevo inserta y responde 201. La columna `xmax` distingue ambos casos sin una consulta adicional.

Contrato de la API

Método y ruta	Entrada	Respuesta	Uso
`GET /api/health`	—	`200 { ok, db, hora }` o `503`	Verificar API y conexión a PostgreSQL.
`GET /api/reportes`	—	`200 { total, reportes }`	Listar los últimos 500 reportes sincronizados.
`POST /api/reportes`	JSON con `id`, `cliente`, `servicio`, `descripcion` y campos opcionales	`201` si inserta, `200` si actualiza, `400` si falta un dato	Guardar de forma idempotente.

Modelo de datos

Campo	Tipo	Regla
`id`	UUID	Clave primaria generada en el cliente.
`tecnico`	TEXT	Opcional en la base; obligatorio en el formulario.
`cliente`	TEXT NOT NULL	Obligatorio.
`servicio`	TEXT NOT NULL	Obligatorio.
`descripcion`	TEXT NOT NULL	Obligatorio, máximo 4000 caracteres.
`ubicacion`	TEXT	Referencia escrita opcional.
`lat`, `lng`	DOUBLE PRECISION	Nulos si no se capturó GPS.
`estado`	TEXT NOT NULL	Valor de negocio; por defecto `reportado`.
`fecha_creacion`	TIMESTAMPTZ	Hora del cliente o `now()` como respaldo.
`actualizado_en`	TIMESTAMPTZ	Se actualiza en cada `ON CONFLICT`.

Topología de despliegue

Entorno	Frontend	API	Base de datos	HTTPS
Local	`vite preview` en `:5173` dentro de Docker	Express en `:3000`	PostgreSQL 16 en `:5433`	`localhost` exento para el Service Worker.
Móvil de prueba	Ngrok apuntando a `:5173`	La misma API por proxy `/api`	La misma base local	Túnel HTTPS de Ngrok.
Producción	Vercel	Render	Neon	TLS de cada proveedor.

En producción, el frontend usa `VITE_API_URL` para apuntar a Render. En local, el proxy de Vite evita CORS y permite un solo túnel de Ngrok para la app y la API.

---

8. Atributos de calidad

Seguridad. El Service Worker y la geolocalización exigen HTTPS fuera de `localhost`. La API limita CORS a los orígenes declarados en `CORS_ORIGIN`, valida el UUID y recorta los textos, usa consultas parametrizadas y limita el cuerpo JSON a 64 kB. Las credenciales viven en variables de entorno del servidor; el frontend no contiene secretos.

Rendimiento. El build precachea 11 entradas, alrededor de 250 KiB, incluida la fuente IBM Plex Sans de 45 KiB. No hay runtime de framework. Lighthouse 11 dio 100 en PWA, accesibilidad, buenas prácticas, rendimiento y SEO tanto en local como sobre la URL pública de Vercel. El reporte público está en `docs/evidencias/lighthouse-produccion.html`.

Disponibilidad. El shell se sirve desde caché cuando no hay red. La API puede no estar disponible sin bloquear la captura: los reportes quedan en `pending`. En Render, el plan gratuito suspende el servicio por inactividad; el primer `POST` puede tardar en despertar. La cola reintenta y el usuario

conserva el dato.

Accesibilidad. Todas las etiquetas están asociadas a su campo, el foco cambia a la vista activa, los estados usan `aria-live` y los controles principales miden al menos 44 px. El contraste cumple AA. Lighthouse de accesibilidad dio 100 en local y en producción.

Mantenibilidad. El frontend separa UI (`main.js`), persistencia (`db.js`) y sincronización (`sync.js`). El Service Worker es un archivo propio y legible. El backend tiene una ruta, una conexión y una migración. La deuda conocida está documentada: no hay autenticación, adjuntos ni resolución de conflictos.

Observabilidad. `GET /api/health` comprueba la base de datos. El cliente guarda `lastError` por reporte y el servidor registra errores no controlados. No hay métricas, trazas ni alertas; para el alcance académico, la salud de la API y el estado visible de la cola son suficientes.

---

9. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación implementada	Riesgo residual
Safari purga IndexedDB tras días sin uso (ITP)	Pérdida de reportes no sincronizados	Instalar la PWA, sincronizar pronto y ofrecer botón manual	Si el usuario nunca abre ni sincroniza, el dato local puede perderse.
iOS no soporta Background Sync	La cola no se envía sola en segundo plano	Evento `online`, sincronización al arrancar y botón manual	La app debe abrirse para sincronizar.
Reintento tras respuesta perdida	Reporte duplicado	UUID del cliente y `ON CONFLICT (id) DO UPDATE`	Ninguno en la operación normal.
API caída o lenta	Reporte sin sincronizar	Cola local y reintento posterior	El reporte permanece pendiente hasta que haya red.
Error 4xx por datos inválidos	Reintento infinito	Estado `error` y mensaje visible, sin reintento automático	Requiere corregir el dato o descartar el reporte.
Service Worker con versión anterior	El usuario ve una versión vieja tras desplegar	`autoUpdate`, `skipWaiting` y `clients.claim`	Puede requerir una recarga adicional; está documentado.
Arranque en frío de Render	Primera petición lenta	Estado de envío y reintento; el shell ya está en caché	La sincronización tarda más, no se pierde.
CORS mal configurado	La app no puede enviar	Lista explícita en `CORS_ORIGIN` por entorno	Un despliegue nuevo debe actualizar la variable.
Permiso de GPS denegado	Reporte sin coordenadas	Campo de referencia escrita y mensaje de estado	La ubicación depende del técnico.
Almacenamiento del navegador borrado	Pérdida de pendientes	Aviso, instalación y sincronización temprana	No hay respaldo de datos no sincronizados.

---

10. Alcance y limitaciones

Incluido: captura offline, listado local, GPS, instalación, sincronización idempotente, API REST, base de datos, entorno Docker, auditoría local y auditoría pública.

Fuera de alcance: autenticación y multiusuario, adjuntos o fotografías, notificaciones push, edición de reportes sincronizados, resolución de conflictos, paginación más allá de 500 filas, analítica y monitoreo centralizado.

Dos límites deben decirse sin adornos. Primero, la app no ofrece un respaldo de los reportes que aún no se sincronizaron: si el usuario borra los datos del navegador, esos reportes se pierden. Segundo, la sincronización en iOS no ocurre en segundo plano; depende de abrir la app, del evento `online` o del botón manual. Ambas limitaciones son consecuencias del entorno web y se mitigan con instalación, avisos y sincronización temprana, no con una promesa que el navegador no puede cumplir.

---

11. Trazabilidad con la rúbrica

Criterio del proyecto	Implementación	Evidencia
Análisis y decisión arquitectónica	Este documento y la matriz ponderada	`docs/dia-01-arquitectura.md`
Compatibilidad e instalación	Tabla iOS/Chromium y checklist	`docs/dia-02-compatibilidad.md`
Entorno reproducible	Tres servicios en Docker Compose	`docker-compose.yml`
API y persistencia	Express, `pg` y PostgreSQL	`backend/src/routes/reportes.js`, `backend/sql/schema.sql`
Interfaz de campo	Formulario, GPS y estados de red	`frontend/index.html`, `frontend/src/main.js`
PWA instalable	Manifest, íconos y prompt	`frontend/public/manifest.webmanifest`, `frontend/src/main.js`
Offline	Service Worker y Dexie	`frontend/src/sw.js`, `frontend/src/db.js`
Sincronización	Cola idempotente y evento `online`	`frontend/src/sync.js`
Auditoría y despliegue	Lighthouse público 100 y producción en Vercel + Render + Neon	`docs/evidencias/lighthouse-produccion.html`, `docs/despliegue.md`

---

12. Conclusión

La decisión no es "usar PWA porque está de moda". El requerimiento R1 elimina la web tradicional y el cliente SOA puro; R3 y R4 eliminan la nativa; la multiplataforma conserva el coste de dos ecosistemas y la distribución por tienda. La PWA cumple R1–R7 con una base de código y un despliegue por URL.

El precio de esa elección es conocido: acceso parcial a las APIs del móvil y dependencia del ciclo de vida del navegador. El proyecto solo necesita GPS, y la sincronización se resuelve con el evento `online`, el arranque y un botón manual, un flujo que no depende de Background Sync y que está previsto para iOS. La arquitectura no oculta esas limitaciones; las convierte en decisiones explícitas.

El reporte se guarda antes de tocar la red. Esa frase resume el diseño y la prueba que debe pasar el prototipo.